

สรุปงาน Cleanup — rag_service/

branch: chore/rag-service-cleanup → merge เข้า main แล้ว (d226bef) วันที่: 15 สิงหาคม 2569
ขอบเขต: ลบ dead code + แก้คอมเมนต์ให้ตรงความจริง — ไม่ปรับโครงสร้าง ไม่เปลี่ยนชื่อ ไม่เพิ่ม abstraction

1. ภาพรวม

หัวข้อ	จำนวน
commit (ไม่รวม merge)	6
ไฟล์ที่ถูกแตะ	21
ลบทั้งไฟล์	4 ไฟล์ / 361 บรรทัด
รวมบรรทัดที่ลบ	638
รวมบรรทัดที่เพิ่ม (ส่วนใหญ่คือคอมเมนต์ที่แก้ให้ถูก)	205
สุทธิ	-433 บรรทัด

ผลทดสอบหลัง merge: rag_service 32 passed · backend 171 passed + 26 subtests ด้านตรวจทุก commit: python -c "import main" และ python -c "from RAG import GraphRAGAgent" ผ่านทั้งคู่

เหตุผลตั้งต้น

โค้ดสะสมสลายอย่างจากการ refactor หลายรอบ: เส้นทางที่ไม่มีใครเรียกถึงแล้ว, flag ของพีเจอร์ที่เลิกใช้, และคอมเมนต์ที่บรรยายพฤติกรรมเก่า งานนี้แบ่งเป็น 2 เฟส — เฟส 1 สัรวจอย่างเดียวไม่แก้โค้ด (ผลลัพธ์คือ CLEANUP_INVENTORY.md) เฟส 2 ค่อยลบจริง

การแยกเฟสคุ่มมาก เพราะเฟส 1 พบว่าสามในสี่ข้อสมมติฐานตั้งต้น ไม่ตรงกับโค้ดจริง ถาลบเลยตั้งแต่แรกจะพังสามจุด (ดู §5)

2. ลบอะไรบ้าง

2.1 ลบทั้งไฟล์

ไฟล์	บรรทัด	เหตุผล
pipeline/thanoy_client.py	105	client ของ Thanoy (AI กฎหมายไทย จาก iApp) ไม่มีใคร import เลยทั้ง repo — ไม่อยู่ใน pipeline/__init__.py ไม่มี router เรียก เขียนไว้สำหรับส่วน "คำแนะนำทางกฎหมาย" ของรายงานที่ backend ไม่เคยสร้างขึ้นมา
howtothanoy.md	115	คู่มือตั้งค่าของ client ข้างบน
app/_perf_probe.py	141	สคริปต์วัด perf ที่ระบุตัวเองว่า throwaway ไม่อยู่ใน allowlist COPY ของ Dockerfile จึงไม่เคยขึ้น production และไปเรียก GraphRetriever._expand_single ซึ่งเป็น private

2.2 เส้นทาง chain ที่ไม่มีใครเรียกถึง

QueryRequest.use_agent มีค่า default เป็น True และ backend/app/services/chat/rag_client.py ก็ hard-code use_agent=True ทุกครั้งที่เรียก แปลว่าครึ่ง chain ของ query_rag() ไม่มีทางถูกเรียกถึง

- ลบ chain branch ออกจาก POST /query
- ลบ print("Agent requested") ที่หลงเหลือ
- ลบ rag_chain ออกจาก startup ใน app/main.py
- ลบ key rag_chain ออกจาก /health
- ลบ re-export ChainResponse / GraphRAGChain ออกจาก __init__.py 3 ไฟล์

ผลพลอยได้ที่ใหญ่กว่าที่คาด: เดิม service สร้าง ทั้ง GraphRAGChain และ GraphRAGAgent ตอน boot แต่ละตัวเปิด Neo4j driver ของตัวเอง, Qdrant client ของตัวเอง และ chat model อีกคู่หนึ่ง ตอนนี้เหลือชุดเดียว

2.3 Ollama / use_local ออกจากเส้นทางที่ให้บริการ

USE_LOCAL default เป็น false และไม่ได้ตั้งไว้ทั้งใน docker-compose.yml และ deploy.yml แปลว่า service ไม่เคยสร้าง ChatOllama เลย

ลบออกจาก: GraphRAGAgent , ContextEvaluator , app/main.py (รวม llm_mode ที่พิมพ์ว่า "LOCAL (Ollama)" ซึ่งไม่มีทางเป็นจริง) และ config USE_LOCAL

2.4 ค่าคงที่และ schema ที่ตายแล้ว

สิ่งที่ลบ	ที่อยู่	เหตุผล
<code>CHROMA_DIR</code> , <code>CHROMA_HOST</code> , <code>CHROMA_PORT</code> , <code>CHROMA_SSL</code> , <code>CHROMA_API_KEY</code> , <code>CHROMA_COLLECTION_*</code>	<code>config.py</code>	ตกค้างจากยุคก่อนย้ายไป Qdrant ไม่มีใครอ่าน
<code>GRAPH_EXPANSION_DEPTH</code>	<code>config.py</code>	ไม่มีใครใช้ — <code>GraphRetriever</code> รับ <code>depth</code> เป็น argument
<code>THANOTY_API_KEY</code> , <code>THANOTY_API_URL</code> , <code>THANOTY_TIMEOUT</code> , <code>THANOTY_ENABLED</code>	<code>config.py</code>	ป้อนให้ client ที่ลบไปแล้ว
<code>print(ATTACK_DOMAINS.items())</code>	<code>config.py</code>	debug print ระดับ module — ยิ่งทุกครั้งที่ import รวมถึงใน container และทุกครั้งที่รัน test
<code>ReviewStatusRequest</code>	<code>schemas/rag.py</code>	ไม่มี router ใช้ — RAG service ไม่มี endpoint review
ตัวแปร <code>gap_warning</code> ที่อ่านมาแล้วไม่ใช่	<code>agent_graph.py</code>	ดู S4

หมายเหตุ: บล็อกคอมเมนต์ `LEGACY` ของ E5 และ mmarco reranker เก็บไว้ เพราะเป็นบันทึกการตัดสินใจ ไม่ใช่โค้ด — ต่างจาก `CHROMA_*` ที่เป็นชื่อจริงที่ import ได้ ซึ่งเป็นช่องทางให้ dead config ถูกหยิบกลับมาใช้โดยไม่ตั้งใจ

3. แก้อะไรบ้าง

3.1 คอมเมนต์ในโค้ด

ที่อยู่	เดิมบอกว่า	ความจริง
<code>cross_lingual.py</code> docstring	pipeline 3 stage: แปล → reason → แปล	stage 1 หายไปจากเส้นทางที่ให้บริการแล้ว เขียนใหม่โดยแยกชัดว่าส่วนไหน agent ใช้ ส่วนไหนเหลือไว้ให้ <code>chain.py</code> + <code>evaluation/</code>
<code>get_fast_system_prompt()</code>	"prompt สำหรับโหมด <code>--fast</code> "	เป็น prompt หลักของ production — <code>SINGLE_CALL_GENERATION</code> เปิดโดย default ทำให้ reasoning node ของ agent ใช้ตัวนี้ คอมเมนต์นี้หลอกที่สุดในโค้ดชุดนี้
<code>agent_graph.py</code> docstring	"Replaces the linear LCEL chain" / "Drop-in companion for <code>GraphRAGChain</code> "	อ้างอิงสิ่งที่ไม่ใช่ของคุณกันแล้ว
<code>_node_reasoning</code> docstring	"Stage 2 ... สังเคราะห์เป็นคำตอบภาษาอังกฤษ"	เส้นทางจริงเขียนภาษาไทยจบในครั้งเดียว
<code>AgentState</code> หัวข้อ — <code>Translation</code> —	क्रम <code>english_query</code>	ไม่มีอะไรถูกแปล — field นี้ copy query เดิมมาตรง ๆ คอมเมนต์ใหม่บอกด้วยว่าทำไมยังใช้ชื่อเดิม (evaluator ผูกกับมันอยู่)
<code>config.py</code> คอมเมนต์ reranker	"ต้องรับคู่ไทย-อังกฤษ ตอน <code>DUAL_QUERY_RETRIEVAL</code> เปิด"	agent ไม่เคยอ่าน flag นั้น — ที่เป็นผู้ข้ามภาษาเพราะ ไม่มีการแปล input เลย ต่างหาก
<code>config.py</code> เส้นทาง STIX	ไม่มีคำอธิบาย	เพิ่มบันทึกว่าทำไมต้องมี existence check (ใน Docker image bundle อยู่ที่ <code>rag_service/</code> แต่ใน repo อยู่ระดับ root)
<code>main.py</code> (CLI)	<code>python main.py --ingest</code>	คำสั่งนี้พังด้วย <code>ImportError</code> — package ใช้ relative import ต้องรันด้วย <code>-m</code>
<code>run_ingest()</code> docstring	"load into Neo4j + ChromaDB "	โหลดเข้า Qdrant
<code>--agent</code> / <code>--fast</code> help	"...with self-reflection and follow-up "	โมดูล follow-up ถูกลบไปแล้ว

3.2 CLAUDE.md

เดิม	แก้เป็น
"LangGraph (agentic loop) + LangChain LCEL"	LangGraph ทำ orchestration, LangChain ให้ abstraction ของ LLM และ message — และระบุชัดว่า LangGraph เป็นคนละ library ไม่ใช่ส่วนหนึ่งของ LangChain ส่วน LCEL เหลือใช้เฉพาะ evaluation
diagram มี stage [CROSS-LINGUAL] Translate query to English	stage นั้นไม่มีจริง วาดใหม่ให้ตรงโค้ด — เพิ่ม [PREPARE] (ตรวจภาษาอย่างเดียว) กับ [DECOMPOSE] + quota retrieval ที่ทำงานจริงแต่ไม่เคยถูกเขียนถึง
--local เป็น flag ของ RAG CLI	ไม่เคยมีอยู่จริง — มีเฉพาะสคริปต์ใน evaluation/
Reasoning LLM = claude-sonnet-4-20250514	default จริงคือ OpenRouter openai/gpt-5.6-luna (ตั้ง CORE_LLM_PROVIDER=anthropic ถึงจะได้ claude-haiku-4-5)
เตือนว่า --ingest หาไฟล์เดอร์ STIX ผิดที่	แก้ไปแล้วในโค้ด — ลบคำเตือนทิ้ง (ดู §5.3)

3.3 เอกสารไทยชุดใหญ่

ARCHITECTURE.md , docs/RAG_Module.md , docs/ARCHITECTURE_v2.md , docs/PRIMER.md มี reference ถึง use_local / Ollama / GraphRAGChain อยู่หลายสิบจุด การเขียนใหม่ทั้งหมดใหญ่เกินขอบเขตงานนี้

เอกสารพวกนี้ประกาศตัวเองว่าถอดมาจากซอร์สโค้ดปัจจุบัน การปล่อยเจียบไว้จึงเป็นทางเลือกที่หลอกกว่า — ใส่ banner ระบุวันที่ตรวจและลิสต์จุดที่ไม่ตรงแล้วแทน (ไฟล์ .pdf ที่ generate คู่กันก็เก่าตามไปด้วย)

4. สิ่งที่น่าสนใจเก็บไว้ (และทำไม)

งานนี้เจอหลายอย่างที่ "ดูเหมือนตาย" แต่จริง ๆ ยังมีคนใช้

chain.py + ครึ่ง translation ของ cross_lingual.py

evaluation/eval_runner.py --mode generation และ evaluation/crosslingual_benchmark.py สร้าง GraphRAGChain ตรง ๆ และเป็น baseline ที่ใช้วัด agent อยู่ ส่วน evaluation/ ถูกกันไว้ไม่ให้แตะเพราะมีงานค้างบน branch eval/real-cti-dataset

วิธีจัดการ: เก็บไฟล์ไว้ แต่ถอดออกจาก pipeline/__init__.py เพื่อไม่ให้เกิดการ import package ลากมันมาด้วย แล้วเขียน header บอกตรง ๆ ว่าเป็น evaluation-only และการปลดระวางต้องย้าย eval_runner มาใช้ GraphRAGAgent ก่อน

stage แปลเป็นไทย (_node_translate_output)

ยังเรียกถึงได้จริง เพราะ _node_reasoning มีทางออก 3 ทาง แต่มีแค่ทางเดียวที่ set answer_is_final :

เส้นทาง	answer_is_final	ผล
query ไทย + reasoning ปกติ	True	ข้าม translate ← เส้นทางหลัก
ACKNOWLEDGE_LIMIT (evaluator ยอมแพ้)	ไม่ถูก set → False	translate ทำงานจริง
ไม่มี reasoning_llm (ไม่มี API key)	ไม่ถูก set → False	translate ถูกเรียกแต่เป็น no-op เพราะ translation_llm ก็เป็น None พร้อมกัน

เส้น ACKNOWLEDGE_LIMIT คั้นข้อความของ evaluator ตรง ๆ โดยไม่เรียก LLM เขียนคำตอบ และข้อความนั้นไม่การันตีว่าเป็นภาษาไทย (system prompt ของ evaluator เป็นอังกฤษ) — stage แปลจึงทำหน้าที่เป็นตาข่ายกันพลาด นอกจากนี้ SINGLE_CALL_GENERATION=False ก็ปลุกเส้นทาง two-stage กลับมาทั้งเส้น

Ollama ใน config + langchain-ollama

ในเส้นทางที่ให้บริการ ไม่เหลือแล้ว แต่ OLLAMA_BASE_URL , LOCAL_LLM_MODEL , LOCAL_EVAL_MODEL , LOCAL_NUM_CTX ต้องอยู่ต่อ เพราะยังมีผู้ใช้ 6 จุดใน evaluation/ + finetune/ :

- generation_metrics.py:152 — OllamaEmbeddings("nomic-embed-text") สำหรับ RAGAS answer_correctness ไม่ได้ gate ด้วย --local (เดิมใช้ OpenAI embeddings แล้วโดน 429 → ได้ nan แถม retry ค้างทั้ง run)
- crosslingual_generation_benchmark.py:398 — local arm ของ A/B เทียบ fine-tune
- chain.py , cross_lingual.py , query_decomposer.py , router.py — branch if use_local: ที่ตอนนี้เปิดได้จาก evaluation/finetune เท่านั้น

ข้อควรรู้: RAGAS judge เป็น cloud อยู่แล้ว (Claude Haiku → OpenRouter) Ollama ที่เหลือทั้งหมดไม่ใช่ judge

อื่น ๆ

MITRE_TABLE_SCORE_THRESHOLD , _TYPE_WEIGHTS , retrieve_multi_quota() — ตามที่สั่งไว้ว่ารอการตัดสินใจ query_fast / query_ultrafast — ยังทำงานได้ เป็นเครื่องมือทดลอง latency โดยตั้งใจ backend/ — ไม่แตะเลยสักไฟล์

5. เรื่องที่พบระหว่างทาง

5.1 มี behaviour change หนึ่งจุด

python -m RAG.GraphRAG.main เปลา่ ๆ เดิมรัน chain ตอนนี้นรัน agent จำเป็นต้องทำ เพราะถ้าไม่ทำ CLI จะกลายเป็นที่เดียวที่ยังลากโมดูล evaluation-only เข้ามาใน app/ ซึ่งขัดกับ header ที่เพิ่งเขียนไว้ใน chain.py เอง ส่วน --agent ยังรับได้แต่เป็น no-op เพื่อไม่ให้ script เดิมพัง

5.2 PARTIAL_ANSWER ไม่ได้ต่อสาย

evaluator มี strategy ตัวที่สามชื่อ PARTIAL_ANSWER (ตอบเท่าที่มี + เตือนว่าข้อมูลขาดตรงไหน) แต่ agent_graph.py ไม่มีที่ไหนเช็ค และ gap_warning ที่อุตสาหกรรมส่งมาก็ถูกอ่านเก็บไว้แล้วไม่ใช้ต่อ — ผู้ใช้ไม่เคยเห็นคำเตือนนั้น

docstring เดิมเขียนว่า *"PARTIAL_ANSWER / ACKNOWLEDGE_LIMIT are then honoured downstream"* ซึ่งจริงครั้งเดียว
งานนี้ลบเฉพาะตัวแปรที่ตายและแก้ docstring ให้ออกตรง ๆ — การต่อสายให้ทำงานจริงเป็น **feature** ไม่ใช่ **cleanup** จึง
ปล่อยไว้ให้ตัดสินใจ

5.3 ปัญหา STIX ingestion แก้ไปแล้ว

CLAUDE.md เตือนว่า `--ingest` หาโฟลเดอร์ STIX จาก `_PROJECT_ROOT` ซึ่งชี้ไป `rag_service/` แต่ bundle อยู่ที่
repo root — ไม่จริงแล้ว `config.py` มี fallback เช็ค existence อยู่ ยืนยันด้วยการ import จริง: `ATTACK_DOMAINS` ชี้ไป
`.../Mitre_ATT&CK Doc/enterprise-attack` ที่มีอยู่จริง

ไม่ต้องแก้โค้ด แค่ลบคำเตือนที่ล้าสมัยและเพิ่มบรรทัดอธิบายว่าทำไมต้องมี fallback

5.4 การ merge กับ main — เกือบพัง 3 จุด

ระหว่างทำงาน `main` เดินหน้าไป 6 commits และ **ลบ chain branch** ออกจาก `/query` ไปเองแล้ว โดยไปไกลกว่า: ตัด
field `answer` ออกจาก HTTP boundary ทั้งหมด (เหลือ `context` + `mitre_table`) และใส่ `extra="forbid"` ทุก
schema

conflict มี 2 ไฟล์ ต้องแก้ด้วยมือ 3 จุด:

1. **auto-merge สร้างบักเงียบ** — git เอา `from typing import Any` ของ branch มารวมกับ
`Literal["completed"]` ของ main → `NameError` ตอน import คือ service ไม่ขึ้นเลย
2. **use_agent ต้องเอากลับมา** — เหตุผลเดิมที่ลบได้คือ pydantic เมิน field แปลกปลอม แต่ `extra="forbid"` ของ
main ทำให้เหตุผลนั้นใช้ไม่ได้ ถ้าปล่อยไปตามเดิม backend ที่ยังส่ง `use_agent: true` จะโดน 422 ทุก request
3. **/health** — main ลบ chain ออกจาก router แล้ว แต่ `/health` ยังอ่าน `app.state.rag_chain` และ
`app/main.py` บน main ก็ยังสร้าง `GraphRAGChain` ทิ้งไว้เฉย ๆ งานนี้ปิดทั้งสองฝั่งให้ตรงกัน

5.5 ปัญหาที่ไม่เกี่ยวกับงานนี้

`cd backend && pytest tests collection` พังทั้ง suite เพราะโฟลเดอร์ `backend/alembic/` (ไม่มี `__init__.py`
ไม่มี `config.py`) ไปบัง package `alembic` ที่ติดตั้งไว้ ทำให้ `from alembic.config import Config` หาไม่เจอ ต้อง
ใช้ `--ignore` ถึงจะรันได้

ยืนยันว่ามีอยู่บน `main` ก่อนแล้ว — branch นี้ไม่มี commit ที่แตะ `backend/` เลย

6. งานที่ยังค้าง

เรื่อง	สถานะ
ลบ <code>chain.py</code> ที่จริง	เลื่อนไว้ตามที่ตัดสินใจ — ต้องย้าย <code>eval_runner --mode generation</code> มาใช้ <code>GraphRAGAgent</code> ก่อน ซึ่งจะทำให้ตัวเลข eval เปลี่ยน
ตัด Ollama ออกจาก <code>evaluation/</code> + <code>finetune/</code>	รอ <code>eval/real-cti-dataset</code> merge ก่อน · ต้องหา embeddings แทน <code>nomic-embed-text</code> (BGE-M3 ที่โหลดอยู่แล้วน่าจะเหมาะสม) และตัดสินใจเรื่อง local arm ของ fine-tune A/B
<code>query_fast</code> / <code>query_ultrafast</code> (~140 บรรทัด)	ยังไม่ตัดสินใจ — เรียกได้จาก CLI เท่านั้น ไม่มี HTTP route ถ้าถือว่าการทดลอง latency จบแล้ว นี่คือการให้อุ่นดับสองที่ตัดได้ และตัดได้แบบไม่กระทบใคร
router ที่ยิง LLM แล้วหาคำตอบ	ยังไม่ตัดสินใจ — <code>_edge_after_route</code> ถูก hard-wire ให้คืน <code>"incident"</code> เสมอ แต่ <code>_node_route_query</code> ยังเรียก LLM ทุก request แล้วโยนผลทิ้ง (มีคอมเมนต์ <code>TEMPORARILY DISABLED ROUTER</code>) การห้ามการเรียกนี้ประหยัด LLM call ได้ 1 ครั้งต่อ request แต่เป็น behaviour change
ต่อสาย <code>PARTIAL_ANSWER</code>	เป็น feature ไม่ใช่ cleanup
แก้ <code>backend/alembic</code> ที่บัง package	ตั้ง task แยกไว้แล้ว

รายละเอียดเพิ่มเติมของการสำรวจเฟส 1 พร้อมหลักฐาน `grep` ว่าใครเรียกใช้อะไรบ้าง อยู่ใน `rag_service/docs/CLEANUP_INVENTORY.md`